

Memory Management with Page Folios

Adrian Huang | May, 2023

- * Based on kernel 6.3 (x86_64) – QEMU
- * 2-socket CPUs (4 cores/socket)
- * 16GB memory
- * Kernel parameter: nokaslr norandmaps
- * KASAN: disabled
- * Userspace: ASLR is disabled
- * Legacy BIOS

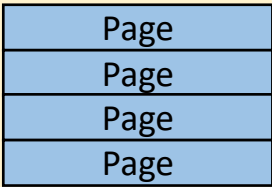
Agenda

- Problem description
 - ✓ [Background] Normal high-order page & compound page
 - ✓ Legacy page cache
- Memory folio's goal
- Solution
 - ✓ Page cache with memory folio
 - ✓ page struct vs folio struct
 - ✓ [Example] `total_mapcount()` implementation difference between legacy approach and folio

Normal high-order page & compound page

```
pages = alloc_pages(GFP_KERNEL, 2);
```

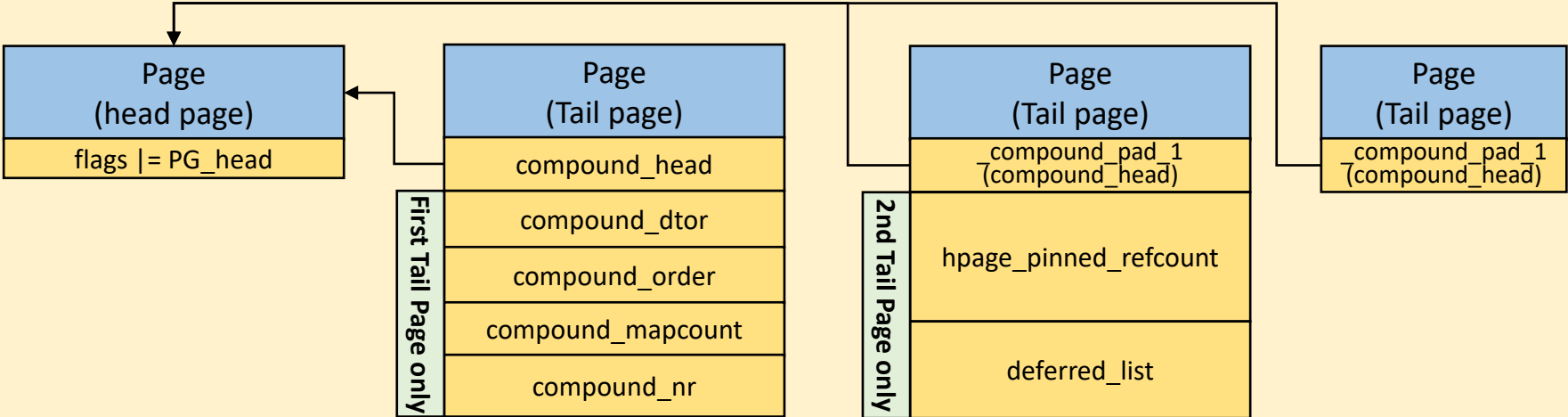
Normal high-order page



Four physically contiguous pages: not a compound page

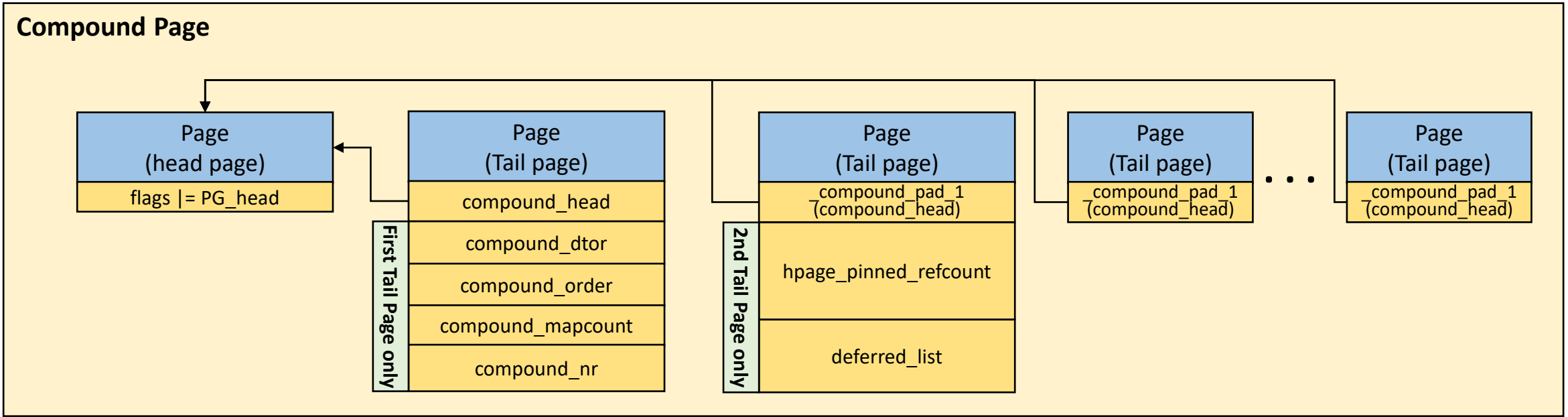
```
pages = alloc_pages(GFP_KERNEL | __GFP_COMP, 2);
```

Compound Page



Four physically contiguous pages: Init compound page metadata during page allocation

Compound page

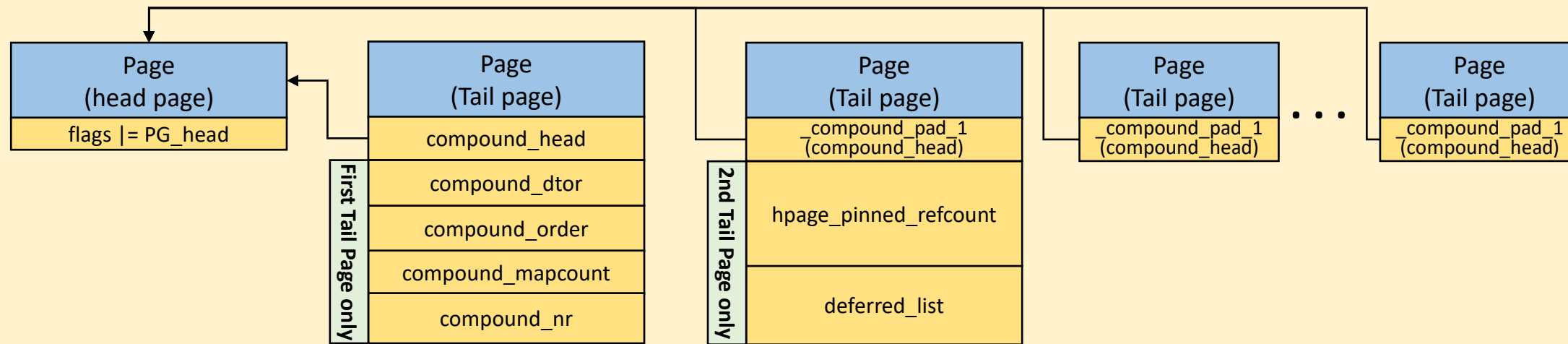


Compound page – Use Cases

- Mainly used in huge page
 - ✓ hugetlbfs (also called HugeTLB Pages or persistent huge pages)
 - Reserved inside the kernel and cannot be used for other purposes.
 - Cannot be swapped out.
 - Two allocation methods:
 - Pre-allocated to the kernel huge page pool with appending kernel parameter.
 - [Dynamically allocated huge pages of the default size] Example: ``echo 10 > /proc/sys/vm/nr_hugepages``
 - User application calls the mmap system call or shared memory system calls (shmget and shmat) to request the huge page allocation.
 - Used by database for many years
 - Manual configuration for hugetlb pages is required.
 - Application change is required. (via open/mmap)

Compound page

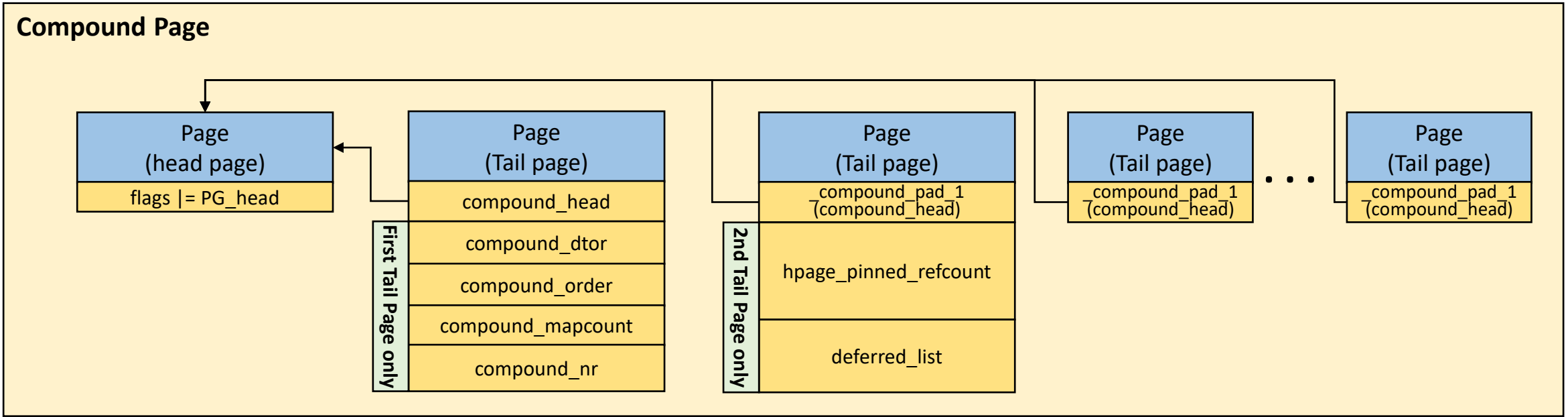
Compound Page



```
adrian@adrian-ubuntu:~$ tree /sys/kernel/mm/hugepages/
/sys/kernel/mm/hugepages/
├── hugepages-1048576kB
│   ├── demote
│   ├── demote_size
│   ├── free_hugepages
│   ├── nr_hugepages
│   ├── nr_hugepages_mempolicy
│   ├── nr_overcommit_hugepages
│   ├── resv_hugepages
│   └── surplus_hugepages
└── hugepages-2048kB
    ├── free_hugepages
    ├── nr_hugepages
    ├── nr_hugepages_mempolicy
    ├── nr_overcommit_hugepages
    ├── resv_hugepages
    └── surplus_hugepages
```

```
adrian@adrian-ubuntu:~$ grep Huge /proc/meminfo
AnonHugePages:      315392 kB
ShmemHugePages:        0 kB
FileHugePages:       0 kB
HugePages_Total:      0
HugePages_Free:       0
HugePages_Rsvd:       0
HugePages_Surp:       0
Hugepagesize:        2048 kB
Hugetlb:              0 kB
```

Compound page

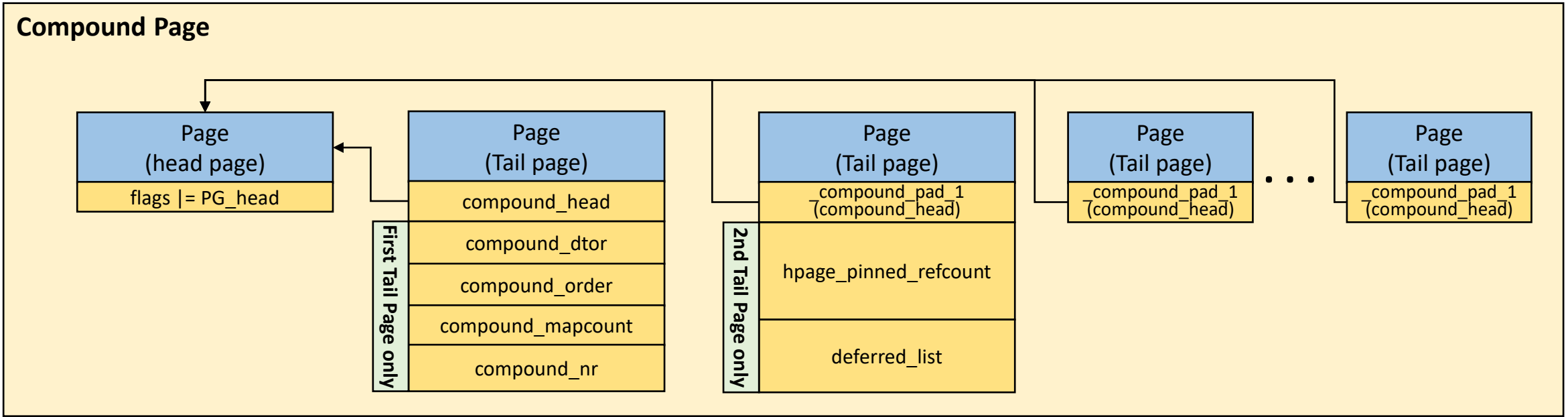


Compound page – Use Cases

- Mainly used in huge page
 - ✓ Transparent Huge Page (THP)
 - Support the automatic promotion and demotion of page sizes
 - Transparent to the application: No need to modify application.
 - Control via `/sys/kernel/mm/transparent_hugepage/enabled`:

```
$ cat /sys/kernel/mm/transparent_hugepage/enabled
always [madvise] never
```

Compound page



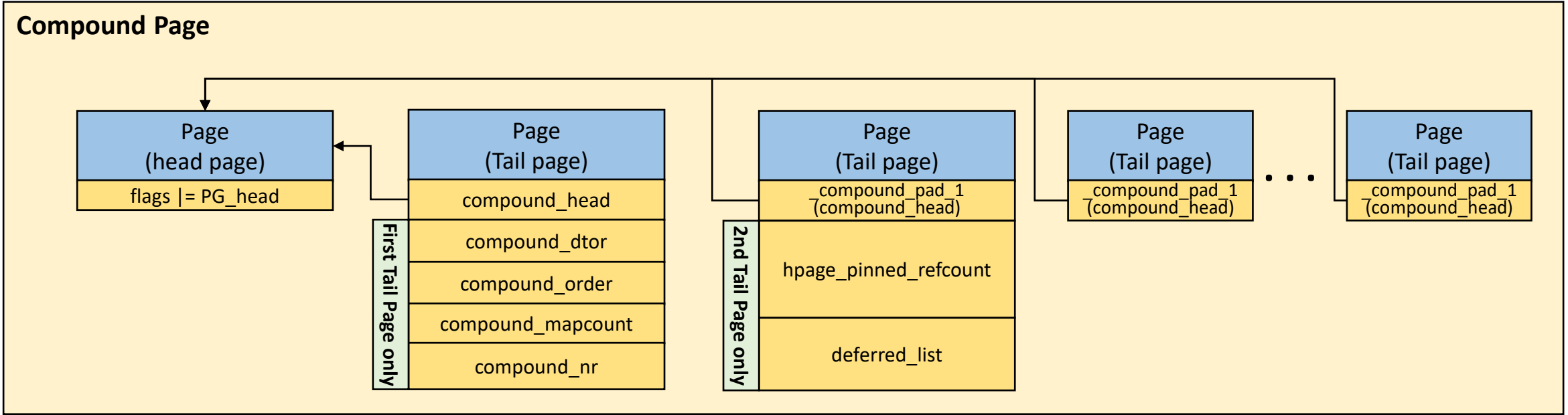
Compound page – Use Cases

- Mainly used in huge page
- kmalloc: allocation size > 8192 bytes
 - Check `kmalloc_order()`
- Memory folio

When to configure compound page?

- Condition: `page order >= 1 && __GFP_COMP` allocation flag is set
- `alloc_pages -> ... -> prep_new_page -> prep_compound_page`

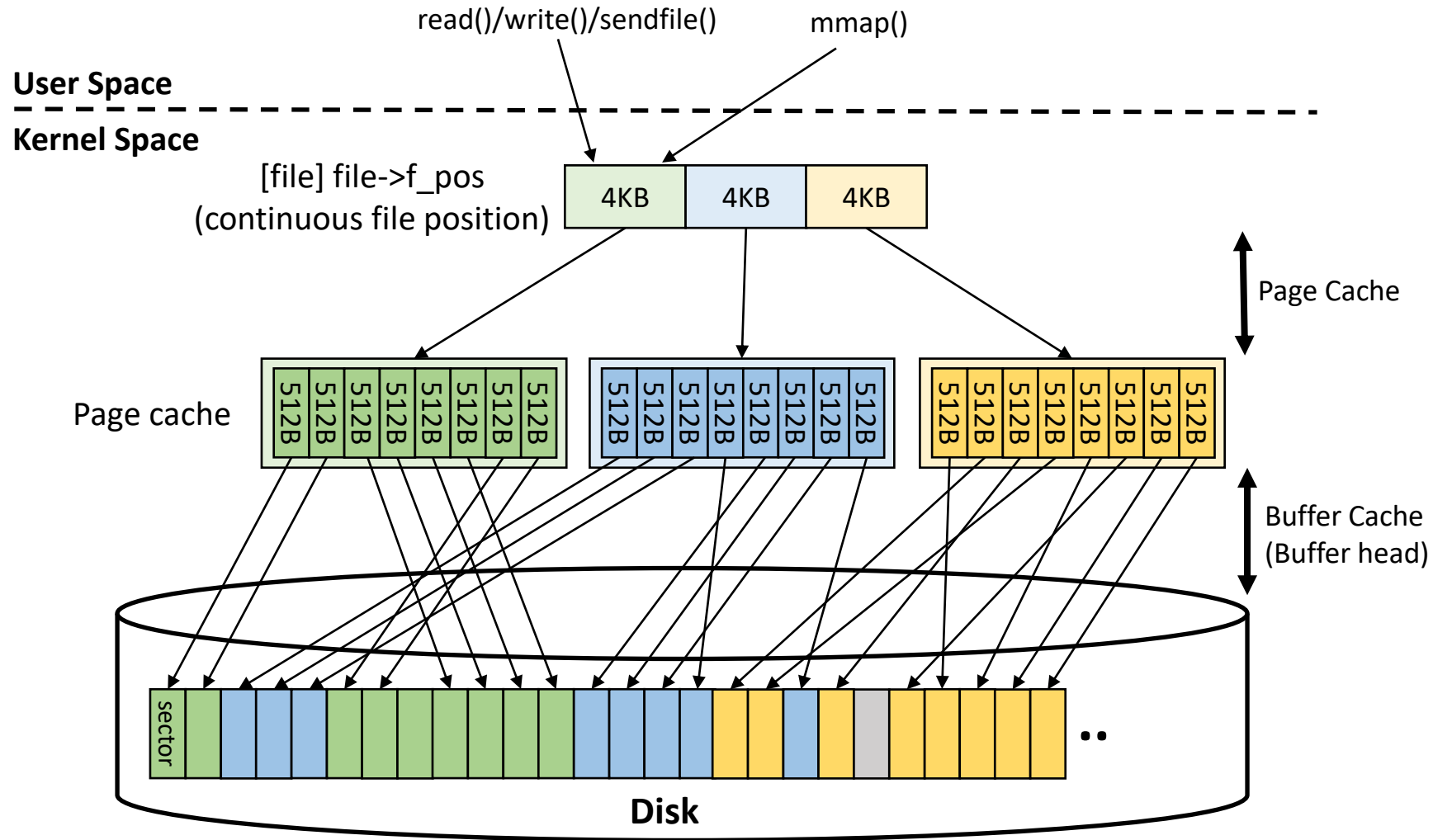
Compound page: Problem Description #1



[Problem Description] No unified interface: Ambiguity

- Some functions may deal with `PAGES_SIZE` unit (4KB): They're unaware of compound pages and huge pages
- Some functions accept the page head **only**
- Some functions accept the page head or page tail
 - ✓ Call `compound_head()` to get the page head: waste instructions to get the page head → **Performance impact**
 - ✓ `compound_head()` users:
 - `get_page()`: This function is called quite frequently
 - `put_page()`: This function is called quite frequently
 - ...

Legacy page cache



Legacy page cache: Problem Description #2

```
$ grep -i active /proc/meminfo
...
Active:                72402996 kB
Inactive:              35386496 kB
Active(anon) :         4524 kB
Inactive(anon) :    1636788 kB
Active(file) :        72398472 kB
Inactive(file) :    33749708 kB
...
```

1. Page cache occupies most of memory pages!!!
2. Each page cache (no “compound page” concept) is added to active/inactive lru list
 - Long lru list: lock contention & cache misses

Agenda

- Problem description
 - ✓ Normal high-order page & compound page
 - ✓ Legacy page cache
- Memory folio's goal
- Solution
 - ✓ Page cache with memory folio
 - ✓ page struct vs folio struct
 - ✓ [Example] total_mapcount() implementation difference between legacy approach and folio

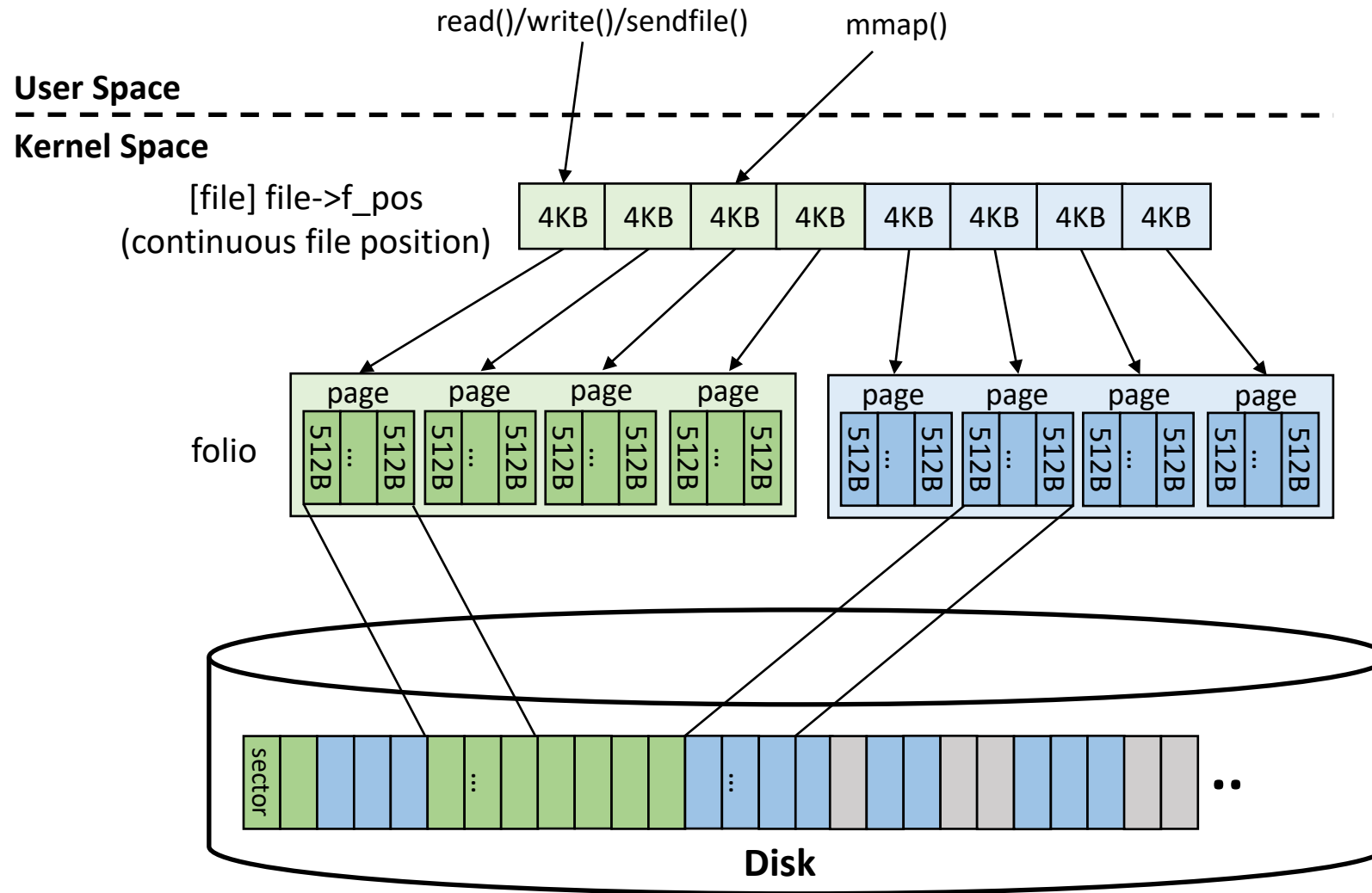
Memory folio's goal

- Unified interface
 - ✓ All accesses via folio struct (~~head page/tail page in a compound page~~)
- [Page Cache] Shorter LRU list
 - ✓ Original: one struct page per 4KB to be added to LRU list
 - ✓ Folio: one struct page (page head) per 8KB, 16KB, 32KB, 64KB, 128KB....and so on (include THP) to be added to LRU list
- [Anonymous page] THP support
 - ✓ `create_huge_pmd -> do_huge_pmd_anonymous_page -> __do_huge_pmd_anonymous_page`

Agenda

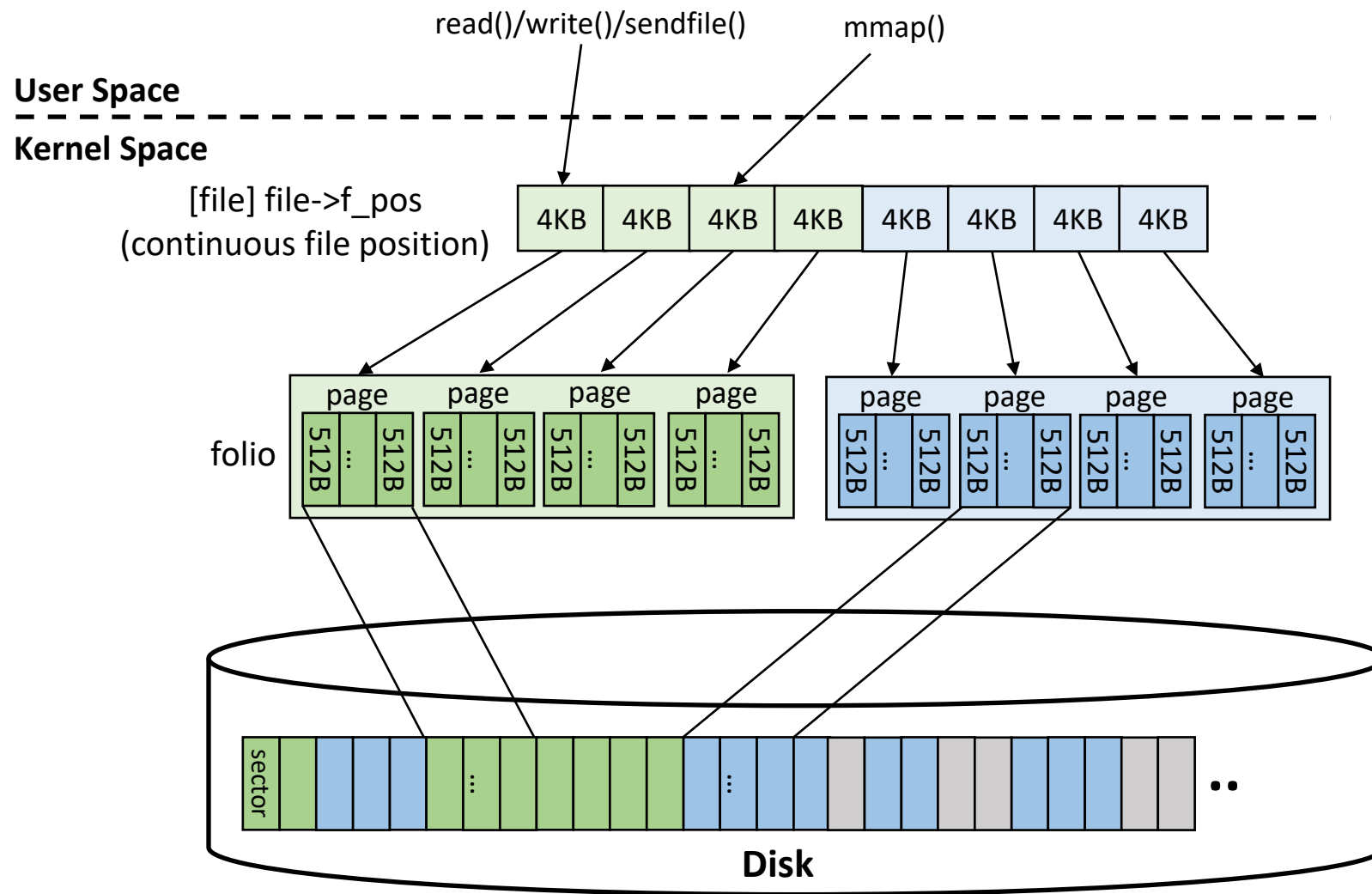
- Problem description
 - ✓ Normal high-order page & compound page
 - ✓ Legacy page cache
- Memory folio's goal
- **Solution**
 - ✓ Page cache with memory folio
 - ✓ page struct vs folio struct
 - ✓ [Example] total_mapcount() implementation difference between legacy approach and folio

Page cache with folio



- **Folio is the container of struct page(s)**
 - ✓ All accesses via folio struct
 - ✓ No tail page → fewer run-time checks

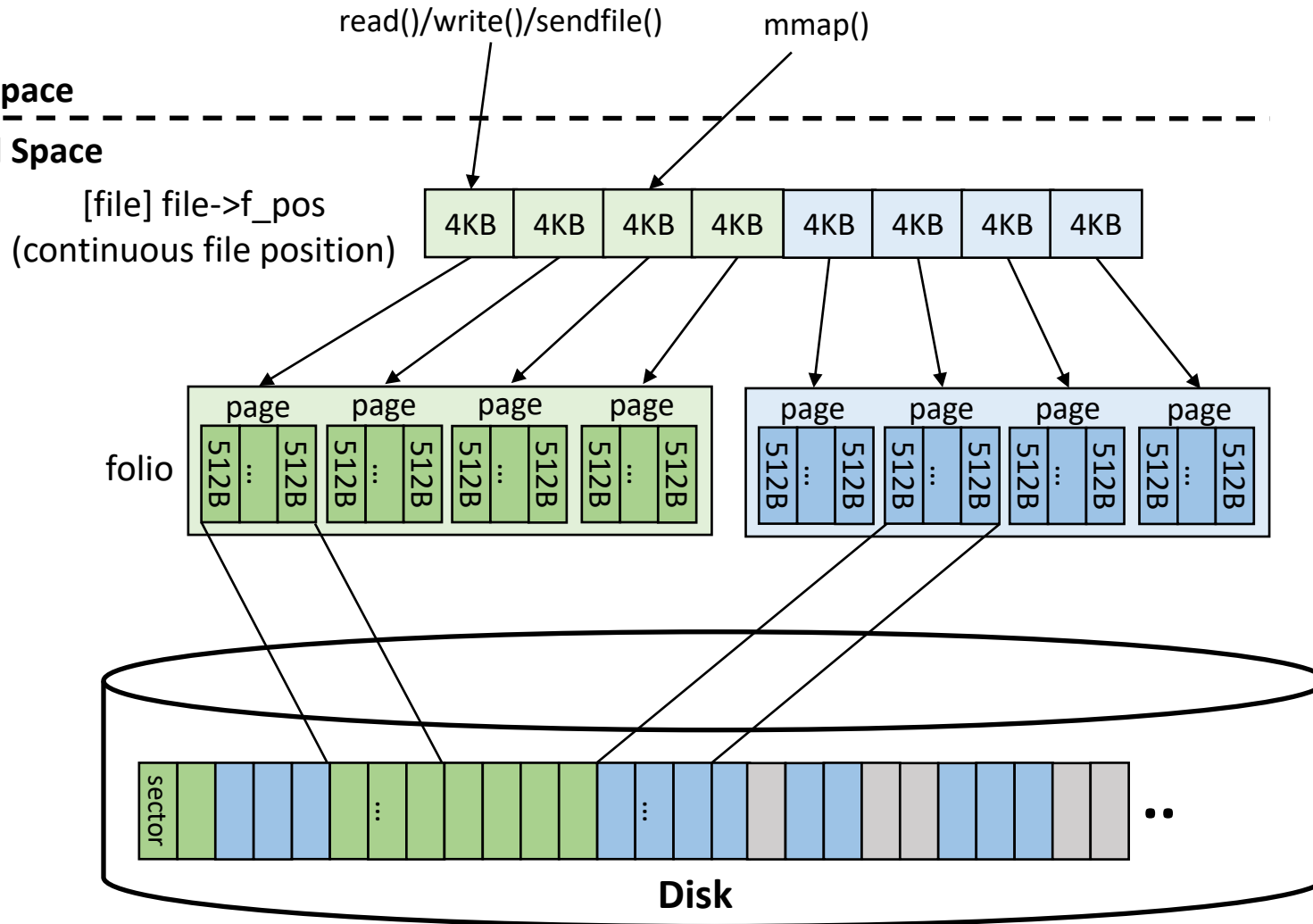
Page cache with folio



Folio's page order: readahead mechanism

- CONFIG_TRANSPARENT_HUGEPAGE is enabled
 - ✓ Minimum: order 2 (4 pages)
 - ✓ Maximum: order 9 (512 pages)
- CONFIG_TRANSPARENT_HUGEPAGE is disabled
 - ✓ Minimum: order 2 (4 pages)
 - ✓ Maximum: order 8 (256 pages)
- Commit 793917d997df ("mm/readahead: Add large folio readahead"): merged in 5.18 kernel
- Default readahead size: 128KB (32 pages)

Page cache with folio



Folio's page order: readahead mechanism

- CONFIG_TRANSPARENT_HUGEPAGE is enabled
 - ✓ Minimum: order 2 (4 pages)
 - ✓ Maximum: order 9 (512 pages)
- CONFIG_TRANSPARENT_HUGEPAGE is disabled
 - ✓ Minimum: order 2 (4 pages)
 - ✓ Maximum: order 8 (256 pages)
- Commit 793917d997df ("mm/readahead: Add large folio readahead"): merged in 5.18 kernel
- Default readahead size: 128KB (32 pages)

1. Short LRU list: Only the head page of folio is added LRU list → Performance improvement
2. 45% improvement for lru-file-mmap-read (vm-scalability): [Matthew Wilcox's PDF file](#)

Page cache with folio: backtrace

* kernel: 6.3

/ # cat /mnt/a.h

```
(gdb) bt
#0  folio_alloc (gfp=gfp@entry=1125450, order=2)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/mempolicy.c:22
#1  0xffffffff810e031d in filemap_alloc_folio (gfp=gfp@entry=1125450,
    order=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/filemap.c:976
#2  0xffffffff810ed2ea in ra_alloc_folio (gfp=<optimized out>,
    order=<optimized out>, mark=<optimized out>, index=0,
    ractl=0xffffc900013dfb30)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/readahead.c:48
#3  page_cache_ra_order (ractl=ractl@entry=0xffffc900013dfb30,
    ra=ra@entry=0xffff8882401b0498, new_order=2)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/readahead.c:53
#4  0xffffffff810ed60c in ondemand_readahead (ractl=0xffffc900013dfb30,
    folio=folio@entry=0x0 <fixed_percpu_data>, req_size=req_size@entry=16)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/readahead.c:68
#5  0xffffffff810ed7d8 in page_cache_sync_ra (
    ractl=ractl@entry=0xffffc900013dfb30, req_count=req_count@entry=16)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/readahead.c:70
#6  0xffffffff810e4aad in page_cache_sync_readahead (req_count=16, index=0,
    file=0xffff8882401b0400, ra=0xffff8882401b0498, mapping=0xffff888241cc0288)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/include/linux/pag
emap.h:1214
#7  filemap_get_pages (iocb=iocb@entry=0xffffc900013dfd50, count=65536,
    fbatch=fbatch@entry=0xffffc900013dfbe8, need_uptodate=true)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/filemap.c:2598
#8  0xffffffff810e4dee in filemap_read (iocb=iocb@entry=0xffffc900013dfd50,
    iter=iter@entry=0xffffc900013dfd28, already_read=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/filemap.c:2693
#9  0xffffffff810e5163 in generic_file_read_iter (
    iocb=iocb@entry=0xffffc900013dfd50, iter=iter@entry=0xffffc900013dfd28)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/mm/filemap.c:2840
#10 0xffffffff812821de in xfs_file_buffered_read (iocb=0xffffc900013dfd50,
    to=0xffffc900013dfd28)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/xfs/xfs_file.c
:277
#11 0xffffffff8128226d in xfs_file_read_iter (iocb=<optimized out>,
    to=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/xfs/xfs_file.c
:302
#12 0xffffffff8119a38c in call_read_iter (iter=0xffffc900013dfd28,
    kio=0xffffc900013dfd50, file=0xffff8882401b0400)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/include/linux/fs.
h:1845
```

```
#13 generic_file_splice_read (in=0xffff8882401b0400, ppos=0xffffc900013dfe08,
    pipe=0xffff8882400f6600, len=<optimized out>, flags=<optimized out>)
--Type <RET> for more, q to quit, c to continue without paging--
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/splice.c:402
#14 0xffffffff81199d24 in do_splice_to (in=in@entry=0xffff8882401b0400,
    ppos=ppos@entry=0xffffc900013dfe08, pipe=pipe@entry=0xffff8882400f6600,
    len=65536, len@entry=16777216, flags=flags@entry=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/splice.c:885
#15 0xffffffff8119a12f in splice_direct_to_actor (
    in=in@entry=0xffff8882401b0400, sd=sd@entry=0xffffc900013dfe48,
    actor=actor@entry=0xffffffff81199a60 <direct_splice_actor>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/splice.c:956
#16 0xffffffff8119a2d9 in do_splice_direct (in=in@entry=0xffff8882401b0400,
    ppos=ppos@entry=0xffffc900013dfeb8, out=out@entry=0xffff8881012de000,
    opos=opos@entry=0xffffc900013dfec0, len=len@entry=16777216,
    flags=flags@entry=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/splice.c:1065
#17 0xffffffff81169f99 in do_sendfile (out_fd=out_fd@entry=1,
    in_fd=in_fd@entry=3, ppos=ppos@entry=0x0 <fixed_percpu_data>,
    count=<optimized out>, count@entry=16777216, max=9223372036854775807,
    max@entry=0)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/read_write.c:1
255
#18 0xffffffff8116b42d in __do_sys_sendfile64 (count=16777216,
    offset=0x0 <fixed_percpu_data>, in_fd=<optimized out>,
    out_fd=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/read_write.c:1
323
#19 __se_sys_sendfile64 (count=16777216, offset=0, in_fd=<optimized out>,
    out_fd=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/read_write.c:1
309
#20 __x64_sys_sendfile64 (regs=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/fs/read_write.c:1
309
#21 0xffffffff8148a1c1 in do_syscall_x64 (nr=<optimized out>,
    regs=0xffffc900013dff58)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/arch/x86/entry/co
mmon.c:50
#22 do_syscall_64 (regs=0xffffc900013dff58, nr=<optimized out>)
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/arch/x86/entry/co
mmon.c:80
#23 0xffffffff8160009f in entry_SYSCALL_64 ()
    at /home/adrian/git-repo/gdb-linux-real-mode/src/linux-6.3/arch/x86/entry/en
try_64.S:120
```

Agenda

- Problem description
 - ✓ Normal high-order page & compound page
 - ✓ Legacy page cache
- Memory folio's goal
- **Solution**
 - ✓ Page cache with memory folio
 - ✓ **page struct vs folio struct**
 - ✓ [Example] `total_mapcount()` implementation difference between legacy approach and folio

page struct vs folio struct

Compound pages

Page #0 (head)
flags
...
Page #1 (tail)
flags
compound_head
compound_dtor
compound_order
compound_mapcount
compound_nr
...
Page #N (tail)
flags
_compound_pad_1
hpage_pinned_refcount
deferred_list

-
-
-

folio				
union	struct	flags		
		union	struct list_head lru	
			struct	void * __filler
				mlock_count
		struct address_space *mapping		
		pgoff_t index		
		void *private		
		atomic_t _mapcount		
		atomic_t _refcount		
		unsigned long memcg_data		
	struct page page			
	union	struct	_flags_1	
_head_1				
unsigned char _folio_dtor				
unsigned char _folio_order				
atomic_t _entire_mapcount				
atomic_t _nr_pages_mapped				
atomic_t _pincount				
unsigned int _folio_nr_pages				
struct page __page_1				
union		struct	_flags_2	
	_head_2			
	void *_hugetlb_subpool			
	void *_hugetlb_cgroup			
	void *_hugetlb_cgroup_rsvd			
	void *_hugetlb_hwpoison			
	struct	_flags_2a		
		_head_2a		
		struct list_head _deferred_list		
		struct page __page_2		

page struct vs folio struct

Compound pages

Page #0 (head)
flags
...
Page #1 (tail)
flags
compound_head
compound_dtor
compound_order
compound_mapcount
compound_nr
...
Page #2 (tail)
flags
_compound_pad_1
hpage_pinned_refcount
deferred_list

•
•
•

folio				
union	struct	union	flags	
			struct list_head lru	
			struct	void * __filler
				mlock_count
		struct address_space *mapping		
		pgoff_t index		
		void *private		
		atomic_t _mapcount		
		atomic_t _refcount		
		unsigned long memcg_data		
		struct page page		
	union	struct	_flags_1	
_head_1				
unsigned char _folio_dtor				
unsigned char _folio_order				
atomic_t _entire_mapcount				
atomic_t _nr_pages_mapped				
atomic_t _pincount				
unsigned int _folio_nr_pages				
struct page __page_1				
union	struct	_flags_2		
		_head_2		
		void * _hugetlb_subpool		
		void * _hugetlb_cgroup		
		void * _hugetlb_cgroup_rsvd		
		void * _hugetlb_hwpoison		
	struct	_flags_2a		
		_head_2a		
		struct list_head _deferred_list		
struct page __page_2				

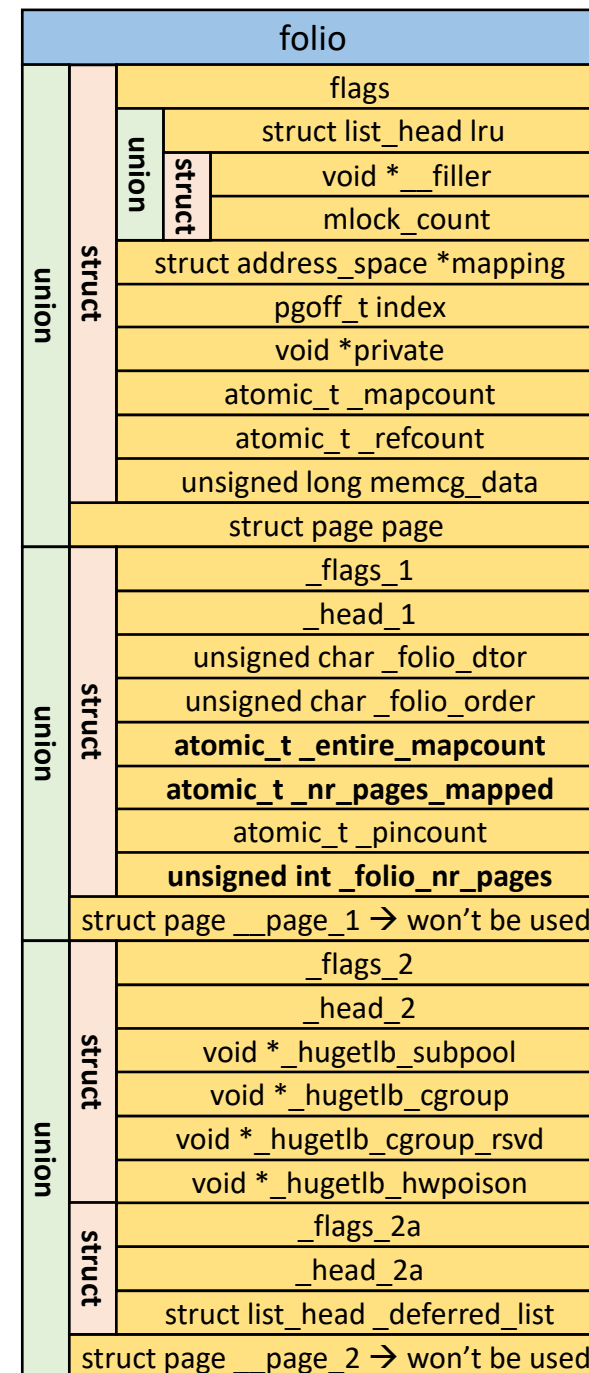
folio's benefit

- [Example] 512KB compound page
 - ✓ page struct: Need to maintain 128 page structs (1 head page and 127 tail pages)
 - ✓ folio struct: Maintain 3 page structs regardless of the size of compound pages.

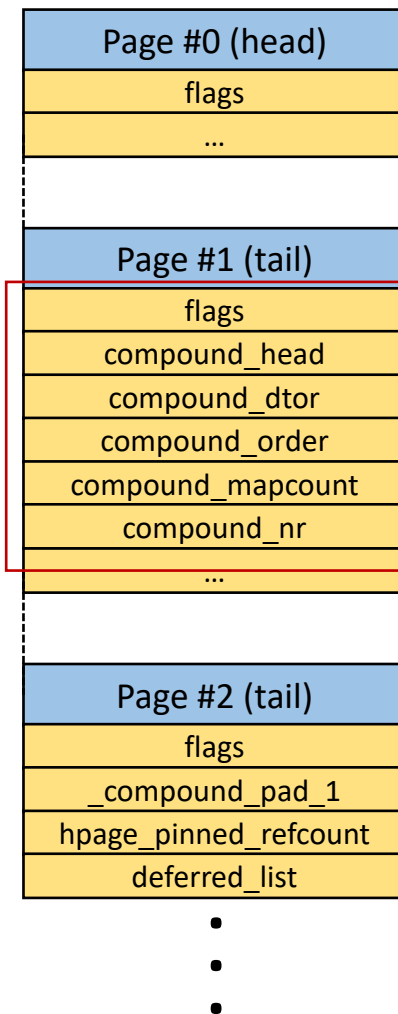
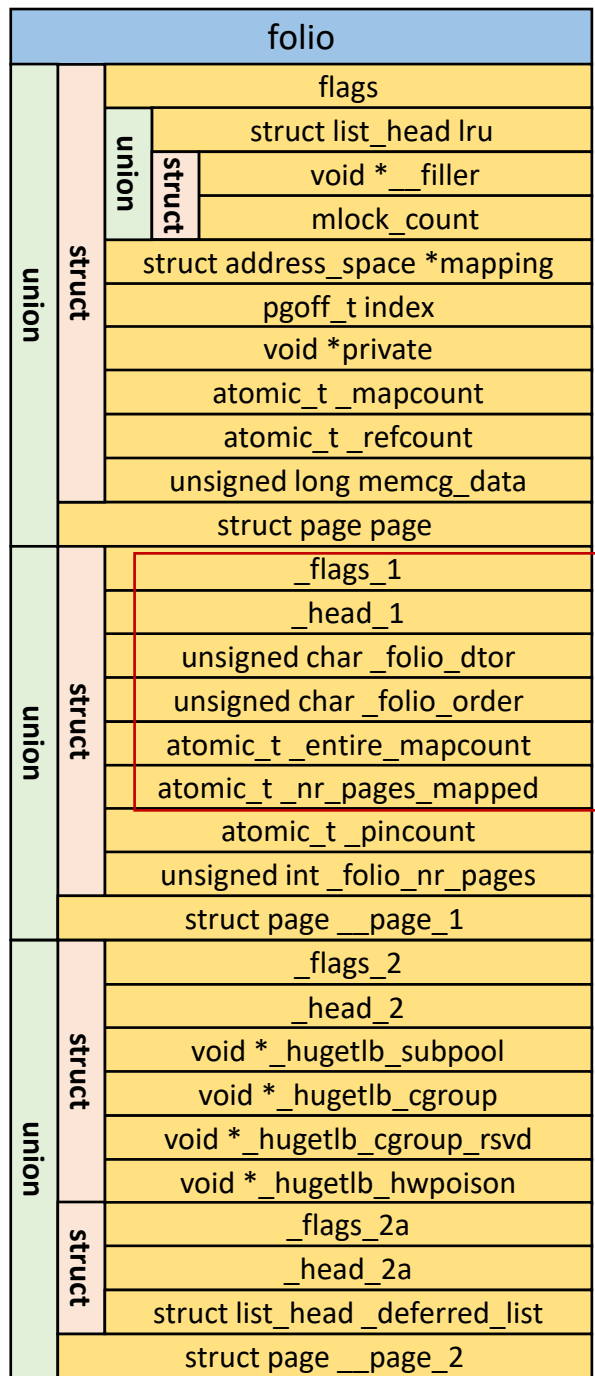
page struct vs folio struct

folio struct's members

- `_entire_mapcount`
 - ✓ The compound page is mapped via a single PMD (huge page).
- `_nr_pages_mapped`
 - ✓ Number of individual subpages (PTE: 4KB pages) are mapped.
 - ✓ Scenario: Two processes map the same memory range
 - ✓ One process maps the entire 2MB compound page (Transparent Huge Page - THP): mapped via a single PMD
 - ✓ The other process maps some 4KB pages within this 2MB memory area: mapped via PTEs
 - ✓ Benefit about THP: No need to split the huge page if other processes map 4KB pages within the same memory area.
- `_folio_nr_pages`
 - ✓ Number of pages in this folio.
 - ✓ `_folio_nr_pages = 1 << order`, where `order > 0`.



folio struct vs legacy compound page



Page #1 of folio and legacy page struct has the same mapping

Agenda

- Problem description
 - ✓ Normal high-order page & compound page
 - ✓ Legacy page cache
- Memory folio's goal
- Solution
 - ✓ Page cache with memory folio
 - ✓ page struct vs folio struct
 - ✓ [Example] `total_mapcount()` implementation difference between legacy approach and folio

[kernel v5.11] total_mapcount()

total_mapcount() users:

- Huge page
- rmap (reverse mapping)

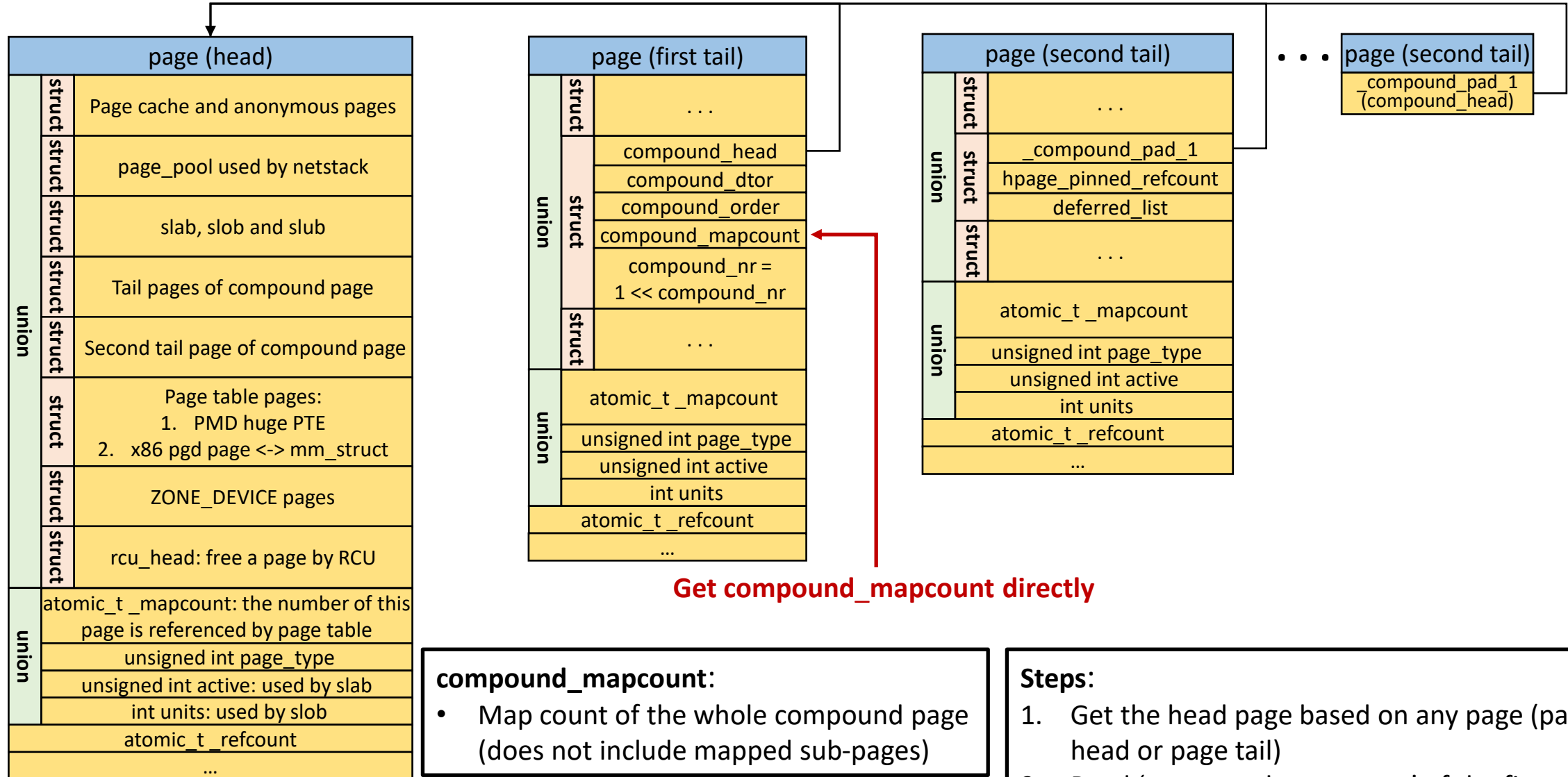
Case 1: Singleton page(s)

page		
union	struct	Page cache and anonymous pages
	struct	page_pool used by netstack
	struct	slab, slob and slub
	struct	Tail pages of compound page
	struct	Second tail page of compound page
	struct	Page table pages: 1. PMD huge PTE 2. x86 pgd page <-> mm_struct
	struct	ZONE_DEVICE pages
	struct	rcu_head: free a page by RCU
union	atomic_t _mapcount: the number of this page is referenced by page table	
	unsigned int page_type	
	unsigned int active: used by slab	
	int units: used by slob	
atomic_t _refcount		
...		

← **Get _mapcount directly**

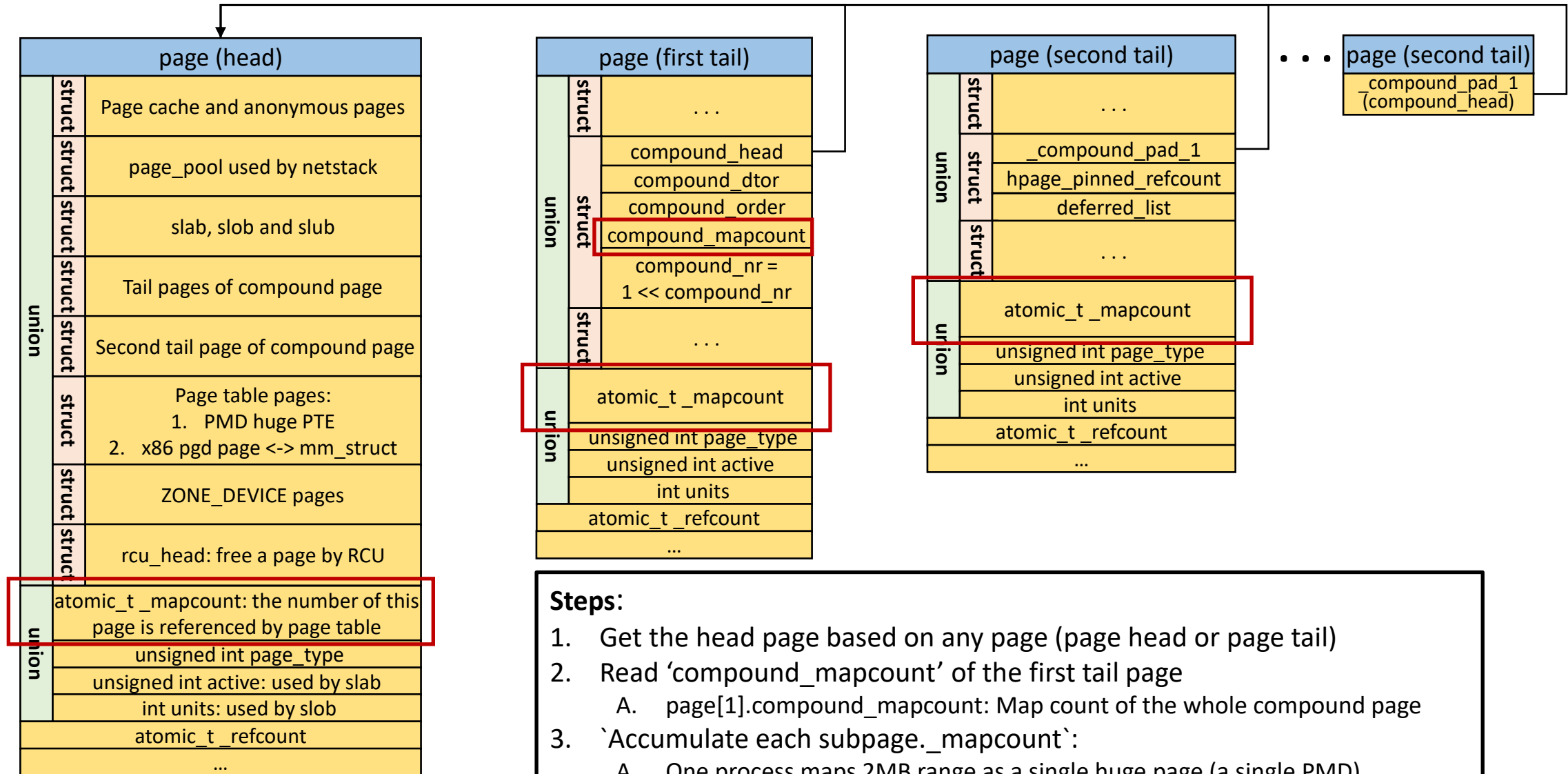
[kernel v5.11] total_mapcount()

Case 2: Compound page & hugetlb (hugetlbfs) page



[kernel v5.11] total_mapcount()

Case 3: [Anonymous page] Compound page & transparent huge page

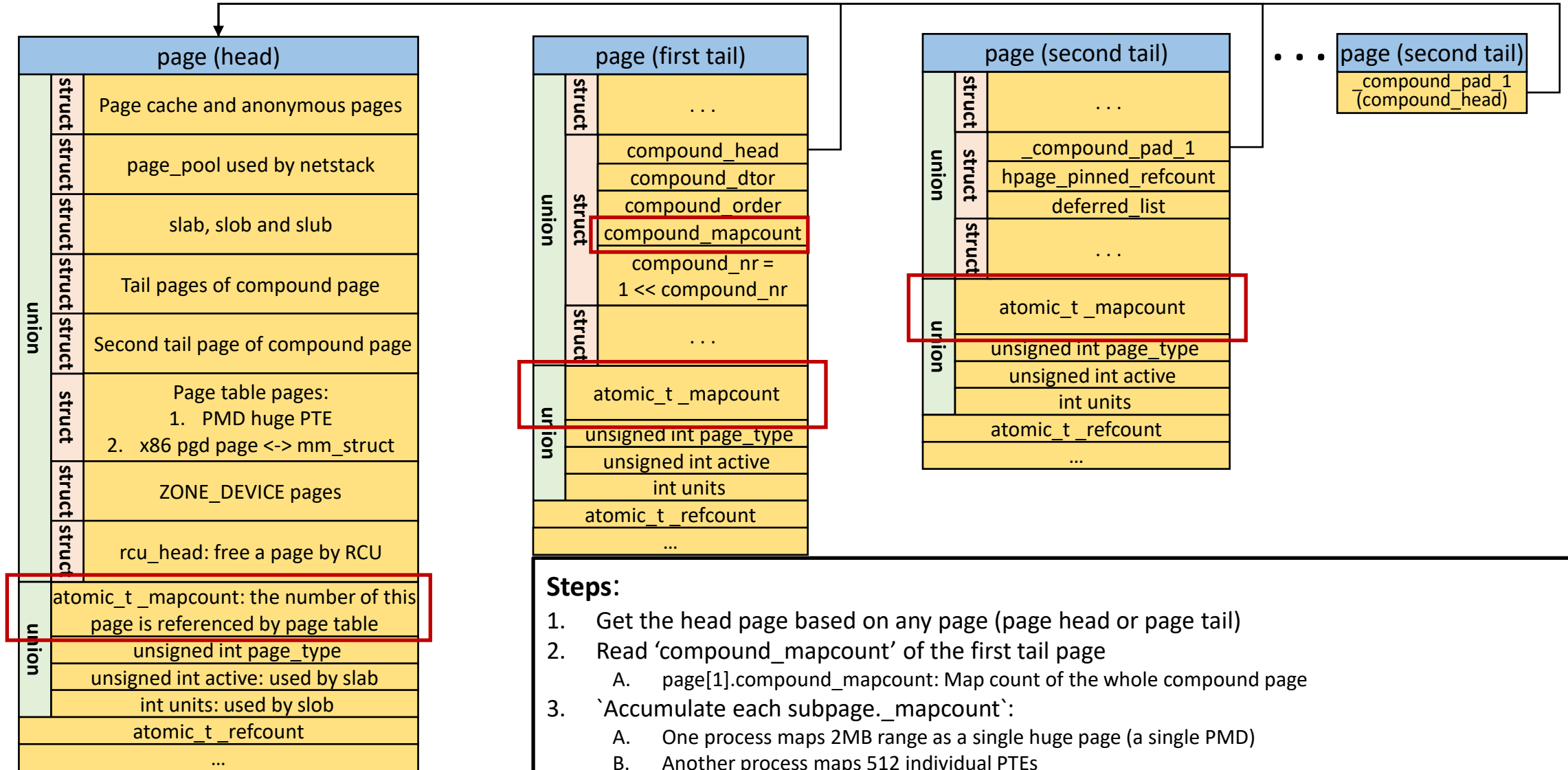


Steps:

1. Get the head page based on any page (page head or page tail)
2. Read 'compound_mapcount' of the first tail page
 - A. page[1].compound_mapcount: Map count of the whole compound page
3. 'Accumulate each subpage._mapcount':
 - A. One process maps 2MB range as a single huge page (a single PMD)
 - B. Another process maps 512 individual PTEs
4. 'Accumulate each subpage._mapcount' + page[1].compound_mapcount

[kernel v5.11] total_mapcount()

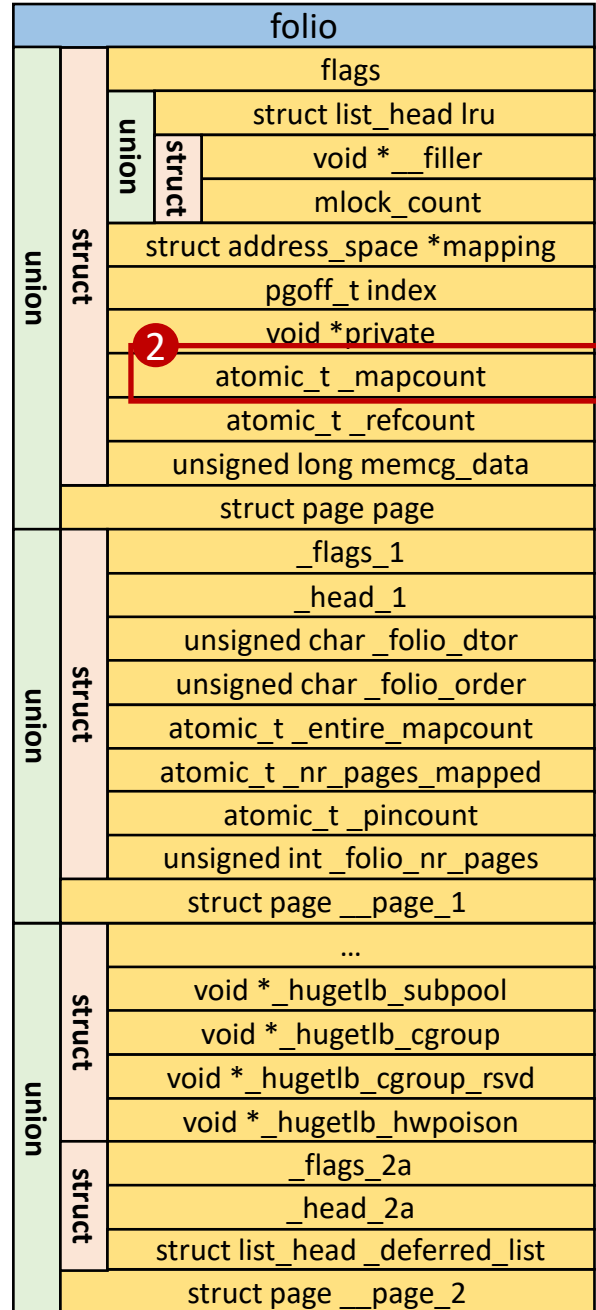
Case 3: [Page cache] Compound page & transparent huge page



Steps:

- Get the head page based on any page (page head or page tail)
- Read 'compound_mapcount' of the first tail page
 - page[1].compound_mapcount: Map count of the whole compound page
- 'Accumulate each subpage._mapcount':
 - One process maps 2MB range as a single huge page (a single PMD)
 - Another process maps 512 individual PTEs
- 'Accumulate each subpage._mapcount' + page[1].compound_mapcount - page[1].compound_mapcount * page[1].compound_nr
 - File pages has compound_mapcount included in _mapcount

[kernel v6.3] total_mapcount() and folio_mapcount()



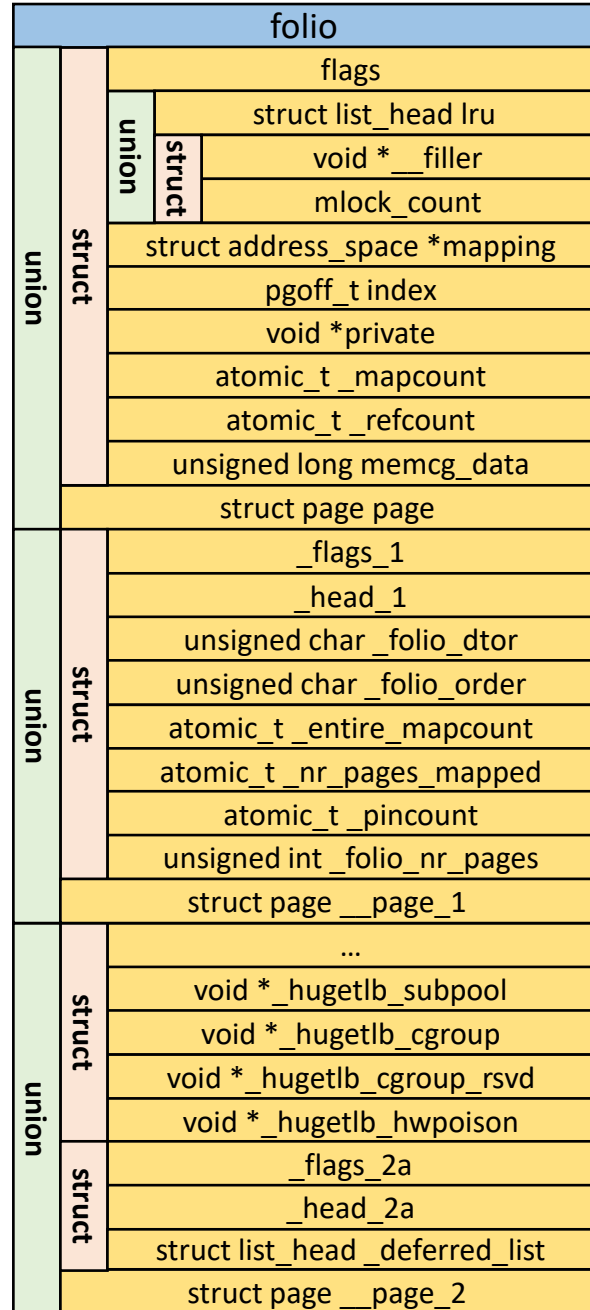
Case 1: Singleton page – Not a compound page

```
static inline int folio_mapcount(struct folio *folio)
{
    if (likely(!folio_test_large(folio)))
        return atomic_read(&folio->_mapcount) + 1;
    return folio_total_mapcount(folio);
}

static inline int total_mapcount(struct page *page)
{
    1 if (likely(!PageCompound(page)))
        return atomic_read(&page->_mapcount) + 1;
    return folio_total_mapcount(page_folio(page));
}

include/linux/mm.h
```

[kernel v6.3] total_mapcount() and folio_mapcount()



← ③ Get _entire_mapcount
← ⑤ Get _nr_pages_mapped

Case 2: Compound page is mapped via PMD (huge page)

```
static inline int folio_mapcount(struct folio *folio)
{
    if (likely(!folio_test_large(folio)))
        return atomic_read(&folio->_mapcount) + 1;
    ① return folio_total_mapcount(folio);
}

static inline int total_mapcount(struct page *page)
{
    if (likely(!PageCompound(page)))
        return atomic_read(&page->_mapcount) + 1;
    return folio_total_mapcount(page_folio(page));
}

include/linux/mm.h
```

```
int folio_total_mapcount(struct folio *folio)
{
    ② int mapcount = folio_entire_mapcount(folio);
    int nr_pages;
    int i;

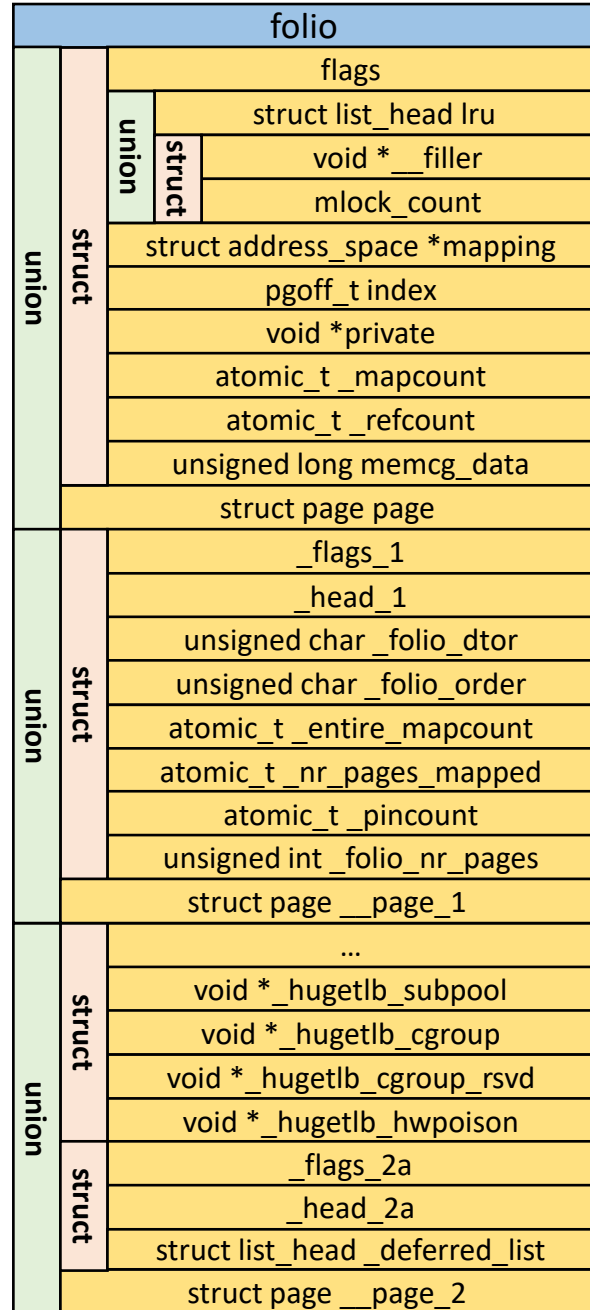
    /* In the common case, avoid the loop when no pages mapped by PTE */
    ④ if (folio_nr_pages_mapped(folio) == 0)
        return mapcount;

    /*
     * Add all the PTE mappings of those pages mapped by PTE.
     * Limit the loop to folio_nr_pages_mapped()?
     * Perhaps: given all the raciness, that may be a good or a bad idea.
     */
    nr_pages = folio_nr_pages(folio);
    for (i = 0; i < nr_pages; i++)
        mapcount += atomic_read(&folio_page(folio, i)->_mapcount);

    /* But each of those _mapcounts was based on -1 */
    mapcount += nr_pages;
    return mapcount;
}

mm/rmap.c 1080,1
```

[kernel v6.3] total_mapcount() and folio_mapcount()



Case 3: Compound page is mapped via PMD (huge page)
and some subpages are mapped by PTE

```
static inline int folio_mapcount(struct folio *folio)
{
    if (likely(!folio_test_large(folio)))
        return atomic_read(&folio->mapcount) + 1;
    1 return folio_total_mapcount(folio);
}

static inline int total_mapcount(struct page *page)
{
    if (likely(!PageCompound(page)))
        return atomic_read(&page->_mapcount) + 1;
    return folio_total_mapcount(page_folio(page));
}

include/linux/mm.h
```

```
int folio_total_mapcount(struct folio *folio)
{
    int mapcount = folio_entire_mapcount(folio);
    int nr_pages;
    int i;

    /* In the common case, avoid the loop when no pages mapped by PTE */
    if (folio_nr_pages_mapped(folio) == 0)
        return mapcount;

    2 /*
     * Add all the PTE mappings of those pages mapped by PTE.
     * Limit the loop to folio_nr_pages_mapped()?
     * Perhaps: given all the raciness, that may be a good or a bad idea.
     */
    nr_pages = folio_nr_pages(folio);
    for (i = 0; i < nr_pages; i++)
        mapcount += atomic_read(&folio_page(folio, i)->_mapcount);

    /* But each of those _mapcounts was based on -1 */
    mapcount += nr_pages;
    return mapcount;
}

mm/rmap.c 1080,1
```

mapcount = folio's _entire_mapcount +
sum(each subpage's _mapcount)

Reference

- [Memory Folios](#)
- [LWN - A memory-folio update](#)
- [LWN - An introduction to compound pages](#)
- [LWN - Huge pages part 1 \(Introduction\)](#)
- Documentation/mm/transhuge.rst

backup

Learn new C standard (C11) from folio: Generic Selection

```
/**
 * folio_slab - Converts from folio to slab.
 * @folio: The folio.
 *
 * Currently struct slab is a different representation of a folio where
 * folio_test_slab() is true.
 *
 * Return: The slab which contains this folio.
 */
#define folio_slab(folio)      (_Generic((folio),
    const struct folio *: (const struct slab *) (folio),
    struct folio *:      (struct slab *) (folio)))
mm/slab.h 78,34
```

6.5.1.1 Generic selection

Syntax

generic-selection:

```
_Generic ( assignment-expression , generic-assoc-list )
```

generic-assoc-list:

```
generic-association
generic-assoc-list , generic-association
```

generic-association:

```
type-name : assignment-expression
default : assignment-expression
```

* Reference from: [ISO/IEC 9899:201x](#)

- C99 defines type-generic macros in the standardized library: the type of argument is detected automatically, and the corresponding function is invoked based on that type.
 - ✓ Example: `sqrt(X)`,
 - X is double → invoke `sqrt()`
 - X is float → invoke `sqrtf()`
 - X is long double → invoke `sqrtl()`
- However, programmers cannot define their own type-generic macros in C99.
- In C11, programmers can define their own type-generic macros:

```
#define sqrt(X) _Generic((X),
    double: sqrt,
    float: sqrtf,
    long double: sqrtl,
    ) (X)
```

Some Functions

```
void lru_cache_add(struct page *page)
{
    struct pagevec *pvec;

    VM_BUG_ON_PAGE(PageActive(page) && PageUnevictable(page), page);
    VM_BUG_ON_PAGE(PageLRU(page), page);

    get_page(page);
    local_lock(&lru_pvecs.lock);
    pvec = this_cpu_ptr(&lru_pvecs.lru_add);
    if (!pagevec_add(pvec, page) || PageCompound(page))
        __pagevec_lru_add(pvec);
    local_unlock(&lru_pvecs.lock);
}
EXPORT_SYMBOL(lru_cache_add);
```

mm/swap.c 445,1

```
void page_cache_ra_order(struct readahead_control *ractl,
                        struct file_ra_state *ra, unsigned int new_order)
{
    +-- 12 lines: struct address_space *mapping = ractl->mapping;-----
    if (new_order < MAX_PAGECACHE_ORDER) {
        new_order += 2;
        if (new_order > MAX_PAGECACHE_ORDER)
            new_order = MAX_PAGECACHE_ORDER;
        while ((1 << new_order) > ra->size)
            new_order--;
    }

    +-- 38 lines: filemap_invalidate_lock_shared(mapping);-----
}
mm/readahead.c 486,1-8
```